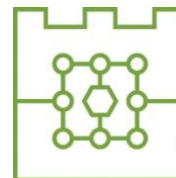




**Politechnika Krakowska
im. Tadeusza Kościuszki**

Wydział Informatyki i Matematyki



Tomasz Umański

Numer albumu: 143033

Tomasz Przegaliński

Numer albumu: 144543

**Porównanie skuteczności metod uczenia głębokiego i lasów losowych w
prognozowaniu naruszeń danych w chmurze**

**Bezpieczeństwo chmur obliczeniowych
na kierunku INFORMATYKA
specjalizacji CYBERBEZPIECZEŃSTWO**

Prowadzący
Andrzej Mycek

Spis treści

1. Wstęp.....	4
1.1 Motywacja	4
1.2 Cel i zakres pracy.....	4
1.3 Znaczenie problemu	4
2. Tematyka i kontekst.....	5
2.1 Chmura obliczeniowa	5
2.2 Modele usług chmurowych	5
2.3 Model współdzielonej odpowiedzialności	5
2.4 Rola uczenia maszynowego w cyberbezpieczeństwie	5
3. Opis problemu.....	5
3.1 Naruszenia danych w chmurze i ich typy	5
3.2 Przykładowy scenariusz naruszenia	6
3.3 Sformułowanie zadania klasyfikacji	6
3.4 Wyzwania	6
4. Uzasadnienie wybranego podejścia	7
4.1 Dlaczego porównanie lasu losowego z sieciami głębokimi.....	7
4.2 Ograniczenia metod sygnaturowych	7
4.3 Las losowy	7
4.4 Sieci głębokie	7
4.5 Miary oceny skuteczności	8
4.6 Przegląd literatury	8
5. Realizacja zadania	8
5.1 Ogólny schemat eksperymentu	8
5.2 Zbiór danych	8
5.3 Przetwarzanie wstępne danych	9
5.4 Konfiguracja modeli.....	10
5.5 Środowisko i narzędzia	10
6. Implementacja programu.....	10
6.1 Organizacja kodu	10
6.2 Konfiguracja eksperymentu	11
6.3 Pozyskiwanie i przygotowanie danych.....	11
6.4 Implementacja modeli.....	11
6.5 Trening, ewaluacja i wizualizacja	12
6.6 Sposób uruchomienia	12
7. Wyniki i analiza	12
7.1 Skuteczność modeli	12
7.2 Krzywe ROC i precyzja-czułość	13
7.3 Analiza błędów	14
7.4 Wydajność i koszt obliczeniowy.....	15
7.5 Przebieg uczenia modeli głębokich	16
7.6 Interpretowalność i ważność cech.....	16
7.7 Synteza wyników	17
8. Dyskusja.....	17
8.1 Skuteczność a koszt w realiach chmury	17
8.2 Interpretowalność i zaufanie	18
8.3 Ograniczenia danych tabelarycznych dla sieci głębokich	18
8.4 Rekomendacje wdrożeniowe	18
9. Wnioski końcowe.....	18
9.1 Podsumowanie.....	18
9.2 Ograniczenia	19

9.3	Podsumowanie końcowe.....	19
10.	Bibliografia	19

1. Wstęp

1.1 Motywacja

Trudno dziś wyobrazić sobie organizację, która nie korzysta z chmury obliczeniowej. Poczta, dokumenty, bazy danych i całe aplikacje coraz częściej działają na serwerach wynajmowanych od dostawców takich jak Amazon Web Services, Microsoft Azure czy Google Cloud. Wraz z usługami do chmury trafiają jednak dane, w tym te najbardziej wrażliwe, co sprawia, że staje się ona łakomym celem dla cyberprzestępców. Naruszenie danych, czyli ich wyciek lub kradzież, należy do najbardziej kosztownych incydentów bezpieczeństwa, a jego wykrycie często następuje dopiero po wielu tygodniach od momentu włamania.

Klasyczne, sygnaturowe systemy ochrony coraz słabiej radzą sobie z nowymi, nieznanymi wcześniej atakami oraz z ogromną skalą i zmiennością ruchu w chmurze. Z tego powodu dużą rolę odgrywają metody uczenia maszynowego, które potrafią samodzielnie uczyć się rozpoznawania złośliwej aktywności na podstawie danych historycznych. Wśród nich szczególne miejsce zajmują dwa podejścia: klasyczne lasy losowe oraz nowsze uczenie głębokie. Oba regularnie osiągają czołowe wyniki, lecz różnią się złożonością, kosztem i łatwością interpretacji decyzji.

1.2 Cel i zakres pracy

Celem pracy jest empiryczne porównanie skuteczności metod uczenia głębokiego i lasów losowych w zadaniu prognozowania naruszeń danych w chmurze. Przez prognozowanie naruszenia rozumiemy rozpoznawanie pojedynczych połączeń sieciowych jako normalnych albo złośliwych, ponieważ wykrycie podejrzanego połączenia stanowi wczesne ostrzeżenie przed pełnym wyciekiem danych.

Praca odpowiada na trzy pytania badawcze:

- Które podejście, las losowy czy sieci głębokie, skuteczniej wykrywa ataki?
- Jaki jest koszt obliczeniowy obu podejść i czy wyższa skuteczność jest warta tego kosztu?
- Czy bardziej rozbudowane sieci (CNN, LSTM) są lepsze od prostej sieci MLP na danych opisujących ruch sieciowy?

1.3 Znaczenie problemu

Skala zjawiska sprawia, że temat jest istotny zarówno z perspektywy biznesowej, jak i inżynierskiej. Pojedyncze naruszenie danych potrafi kosztować organizację miliony złotych, licząc koszty bezpośrednie (analiza incydentu, powiadomienia, kary regulacyjne) oraz pośrednie (utrata reputacji i klientów). Dodatkowo czas od włamania do jego wykrycia bywa bardzo długi, co oznacza, że atakujący mają wiele czasu na wyrządzenie szkód. Każde rozwiązanie skracające ten czas, nawet o kilka godzin, ma więc realną wartość. Automatyczne wykrywanie podejrzanego ruchu jest jednym z najtańszych i najszybciej skalujących się sposobów osiągnięcia tego celu, a jego skuteczność zależy wprost od jakości zastosowanego modelu uczenia maszynowego, co czyni porównanie metod tematem o praktycznym znaczeniu.

2. Tematyka i kontekst

2.1 Chmura obliczeniowa

Chmura obliczeniowa to model dostarczania zasobów informatycznych (mocy obliczeniowej, pamięci, baz danych, aplikacji) jako usługi dostępnej przez sieć. Zamiast utrzymywać własne serwerownie, organizacje wynajmują zasoby od dostawcy i płacą za ich rzeczywiste wykorzystanie. Wygoda i oszczędność mają jednak swoją cenę: dane wielu klientów żyją na współdzielonej infrastrukturze, dostępnej zdalnie z dowolnego miejsca na świecie, a powierzchnia ataku znacząco rośnie.

2.2 Modele usług chmurowych

Chmura dostarczana jest w kilku modelach usługowych, które różnią się zakresem odpowiedzialności klienta. W modelu IaaS (infrastruktura jako usługa) klient otrzymuje maszyny wirtualne i sieć, samodzielnie zarządzając systemem operacyjnym i aplikacjami. W modelu PaaS (platforma jako usługa) dostawca zarządza również środowiskiem uruchomieniowym, a klient skupia się na własnym kodzie. W modelu SaaS (oprogramowanie jako usługa) klient korzysta z gotowej aplikacji przez przeglądarkę. Niezależnie od modelu, komunikacja sieciowa pozostaje wspólnym mianownikiem i nośnikiem śladów potencjalnych ataków, dlatego analiza ruchu sieciowego jest uniwersalnym punktem zaczepienia dla mechanizmów wykrywania naruszeń.

2.3 Model współdzielonej odpowiedzialności

Bezpieczeństwo w chmurze opiera się na tzw. modelu współdzielonej odpowiedzialności. Dostawca chmury odpowiada za bezpieczeństwo samej infrastruktury (centra danych, sprzęt, wirtualizacja), natomiast za poprawną konfigurację usług, kontrolę dostępu i ochronę własnych danych odpowiada klient. W praktyce ogromna część głośnych wycieków danych wynikała nie z włamania do serwerów dostawcy, lecz z błędów po stronie użytkownika: źle ustawionych uprawnień, publicznie dostępnych zasobów czy przejętych kont. To pokazuje, że samo przeniesienie danych do chmury nie zwalnia organizacji z obowiązku ich monitorowania.

2.4 Rola uczenia maszynowego w cyberbezpieczeństwie

Skala i zmienność ruchu sieciowego w chmurze są tak duże, że ręczne pilnowanie bezpieczeństwa przestaje być możliwe. Uczenie maszynowe pozwala zautomatyzować ten proces: model uczy się rozpoznawać wzorce złośliwej aktywności na podstawie historycznych przykładów, a następnie klasyfikuje nowy ruch w czasie zbliżonym do rzeczywistego. W odróżnieniu od metod sygnaturowych, modele uczenia maszynowego potrafią uogólniać i wykrywać również warianty ataków, których wcześniej nie widziały. To właśnie czyni je obiecującym narzędziem w ochronie danych w chmurze i uzasadnia ich porównanie podjęte w tej pracy.

3. Opis problemu

3.1 Naruszenia danych w chmurze i ich typy

Naruszenie danych to sytuacja, w której dane trafiają w niepowołane ręce: zostają wykradzione, ujawnione lub zmienione bez uprawnienia. Konsekwencje bywają dotkliwe, od kar

finansowych (np. na gruncie RODO), przez utratę zaufania klientów, po realne straty operacyjne. W środowisku chmurowym najczęstsze drogi prowadzące do naruszenia obejmują:

- przejście kont, np. zgadywanie haseł metodą brute force do usług logowania;
- eksfiltrację danych, czyli niepostrzeżone wyprowadzanie danych na zewnątrz, często ukryte w pozornie zwykłym ruchu;
- ataki DoS/DDoS, zalewające usługi ruchem, czasem jako zasłona dymna dla innych działań;
- infiltrację i ruch boczny, gdy po zdobyciu przyczółka atakujący przemieszcza się w infrastrukturze ofiary;
- aktywność botnetów oraz ataki na aplikacje webowe (np. SQL injection), również prowadzące do wycieku danych.

Wszystkie te działania mają jedną wspólną cechę: zostawiają ślad w ruchu sieciowym. Analiza tego ruchu pozwala wychwycić podejrzaną aktywność, zanim przerodzi się ona w pełne naruszenie.

3.2 Przykładowy scenariusz naruszenia

Aby zilustrować problem, rozważmy typowy scenariusz. Atakujący najpierw skanuje publicznie dostępne usługi chmurowe ofiary, poszukując otwartych portów. Następnie próbuje uzyskać dostęp do usługi zdalnego logowania, wielokrotnie zgadując hasła (atak brute force). Po przejściu konta loguje się i rozpoczyna rozpoznawanie wewnętrzne, a w końcu kopiuje dane na zewnętrzny serwer. Na każdym z tych etapów powstaje charakterystyczny ruch sieciowy: wiele krótkich połączeń podczas skanowania, liczne nieudane próby logowania, nietypowe połączenia wychodzące podczas eksfiltracji. Model klasyfikujący połączenia ma szansę wychwycić każdy z tych etapów, dając zespołowi bezpieczeństwa czas na reakcję, zanim dojdzie do faktycznej utraty danych.

3.3 Sformułowanie zadania klasyfikacji

Naruszenie danych rzadko jest pojedynczym zdarzeniem; to zwykle finał całego łańcucha działań: rozpoznania, włamania, eskalacji uprawnień i wreszcie kradzieży danych. Wykrycie wczesnych ogniw tego łańcucha pełni rolę prognostyczną, bo pozwala zareagować, zanim dojdzie do wycieku. Dlatego prognozowanie naruszenia utożsamiamy z rozpoznawaniem złośliwych połączeń sieciowych.

Zadanie sprowadza się do prostego pytania zadawanego dla każdego połączenia: czy to zwykły, normalny ruch, czy raczej atak? Każde połączenie opisujemy zestawem liczb, na przykład jak długo trwało, ile pakietów i bajtów przesłano, jak często czy jakie flagi sieciowe się pojawiły. Na tej podstawie model przykleja jedną z dwóch etykiet: ruch normalny albo atak. Jest to więc klasyczne zadanie klasyfikacji binarnej; ponieważ dane zawierają też informację o rodzaju ataku, możliwa jest dodatkowo analiza poszczególnych typów zagrożeń.

3.4 Wyzwania

Wykrywanie naruszeń nie jest proste z kilku praktycznych powodów:

- Ataki są rzadkie. W normalnym ruchu tonie garstka złośliwych połączeń, przez co model łatwo wpada w pułapkę uznawania wszystkiego za normalne i przeocza zagrożenia.
- Nie każdy błąd kosztuje tyle samo. Przeoczenie ataku jest zwykle znacznie groźniejsze niż fałszywy alarm, dlatego zależy nam przede wszystkim na wysokiej wykrywalności.

- Złośliwość kryje się w kombinacjach cech. Pojedyncza cecha rzadko zdradza atak; dopiero ich zestawienie tworzy podejrzany obraz, co wyklucza najprostsze metody liniowe.
- Liczy się szybkość i koszt. W chmurze model trzeba często dotrenowywać i stosować na bieżąco do ogromnych ilości ruchu, więc nie może być zbyt kosztowny obliczeniowo.

4. Uzasadnienie wybranego podejścia

4.1 Dlaczego porównanie lasu losowego z sieciami głębokimi

Spośród wielu metod uczenia maszynowego do porównania wybrano dwie najpopularniejsze i jednocześnie reprezentatywne rodziny. Las losowy to sprawdzona, klasyczna technika, która w zadaniach wykrywania intruzji wielokrotnie osiągała czołowe wyniki przy niewielkim koszcie. Uczenie głębokie reprezentuje natomiast nowoczesny nurt, który zrewolucjonizował analizę obrazów, dźwięku i tekstu. Zestawienie tych dwóch podejść pozwala odpowiedzieć na praktyczne pytanie: czy w ochronie danych w chmurze warto sięgać po skomplikowane sieci neuronowe, czy wystarczy dobrze dobrana metoda klasyczna.

4.2 Ograniczenia metod sygnaturowych

Tradycyjne systemy wykrywania włamań opierają się na sygnaturach, czyli wzorcach znanych ataków. Działają one dobrze, dopóki zagrożenie jest znane, lecz zawodzą wobec nowych wariantów, których sygnatury jeszcze nie powstały. W chmurze, gdzie ruch jest ogromny i zmienny, a atakujący stale modyfikują techniki, podejście sygnaturowe staje się niewystarczające. Uczenie maszynowe uzupełnia tę lukę, ucząc się ogólnych cech złośliwej aktywności zamiast zapamiętywać pojedyncze wzorce, dzięki czemu potrafi rozpoznać również nieznane wcześniej zagrożenia o podobnym charakterze.

4.3 Las losowy

Punktem wyjścia jest drzewo decyzyjne, które można sobie wyobrazić jako listę pytań typu: czy liczba pakietów w połączeniu przekracza pewien próg? Odpowiadając na kolejne pytania, schodzimy w dół drzewa aż do liścia, który mówi, czy to ruch normalny, czy atak. Pojedyncze drzewo jest jednak kapryśne i łatwo się przeucza. Las losowy rozwiązuje ten problem prostą sztuczką: zamiast jednego drzewa buduje ich setki, a każde uczy na nieco innym, losowym wycinku danych i cech. Decyzja końcowa zapada przez głosowanie, dzięki czemu wynik jest znacznie stabilniejszy. Las losowy nie wymaga skalowania danych, dobrze radzi sobie ze złożonymi zależnościami, szybko się uczy i potrafi wskazać, które cechy były najważniejsze przy podejmowaniu decyzji.

4.4 Sieci głębokie

Sieć neuronowa to model luźno inspirowany działaniem mózgu, złożony z warstw neuronów. Każdy neuron łączy informacje z poprzedniej warstwy, mnoży je przez wyuczone wagi i przepuszcza przez nieliniową funkcję aktywacji. Określenie głęboka oznacza, że warstw jest wiele, dzięki czemu sieć stopniowo buduje coraz bardziej złożone wzorce: od prostych zależności po abstrakcyjne sygnatury złośliwej aktywności. Uczenie polega na wielokrotnym pokazywaniu sieci przykładów i korygowaniu wag metodą wstecznej propagacji błędów tak, aby sieć coraz rzadziej się myliła. W pracy porównano trzy architektury:

- MLP (perceptron wielowarstwowy) to najprostsza sieć w pełni połączona, traktująca dane jak zwykłą tabelę liczb; jest naturalnym wyborem dla danych o połączeniach sieciowych.
- CNN 1D (sieć konwolucyjna) przesuwając okienko po danych i wyszukuje powtarzające się, lokalne wzorce; sprawdza się w analizie sygnałów.
- LSTM (sieć rekurencyjna) ma pamięć i jest zaprojektowana do danych o charakterze sekwencji; włączono ją, aby sprawdzić, czy modelowanie sekwencyjności cech przynosi korzyść.

4.5 Miary oceny skuteczności

Sama dokładność, czyli odsetek poprawnych decyzji, bywa myląca, gdy ataki są rzadkie: model uznający wszystko za ruch normalny może mieć wysoką dokładność, a jednocześnie nie wykryć żadnego włamania. Dlatego oceniamy modele kilkoma uzupełniającymi się miarami:

- Precyzja: spośród połączeń oznaczonych jako atak, jaki odsetek faktycznie był atakiem. Niska precyzja oznacza wiele fałszywych alarmów.
- Czułość: spośród wszystkich prawdziwych ataków, ile udało się wykryć. Niska czułość oznacza przeoczone włamania, czyli najgroźniejszy rodzaj błędu.
- F1: jedna liczba godząca precyzję i czułość, wygodna do porównań.
- ROC-AUC i PR-AUC: miary jakości niezależne od ustawionego progu decyzyjnego; im bliżej jedności, tym lepiej.

W kontekście ochrony danych w chmurze szczególnie zależy nam na wysokiej czułości, aby nie przeoczyć naruszenia, przy rozsądnej precyzji, aby nie zasypać analityków fałszywymi alarmami.

4.6 Przegląd literatury

Wykrywanie włamań metodami uczenia maszynowego to dobrze zbadany temat. Powszechnie wykorzystuje się publiczne zbiory danych, takie jak NSL-KDD, UNSW-NB15 czy CIC-IDS2017 i CSE-CIC-IDS2018 przygotowane przez Canadian Institute for Cybersecurity. W wielu badaniach metody oparte na drzewach, jak las losowy, okazują się równie dobre lub lepsze od sieci głębokich, gdy dane mają postać tabeli, a tak właśnie wyglądają cechy połączeń sieciowych. Sieci głębokie błyszczą przede wszystkim tam, gdzie dane są surowe i mają wyraźną strukturę przestrzenną lub czasową. Niniejsza praca sprawdza, czy ta obserwacja potwierdza się również w analizowanym zadaniu chmurowym.

5. Realizacja zadania

5.1 Ogólny schemat eksperymentu

Aby porównanie było uczciwe, wszystkie modele uczono i testowano na tych samych danych oraz oceniano tymi samymi miarami. Eksperyment przebiega według prostego scenariusza: wczytanie danych, ich przygotowanie, wytrenowanie pięciu modeli (regresji logistycznej, lasu losowego oraz sieci MLP, CNN i LSTM), ocena każdego z nich pod kątem skuteczności i kosztu, a na końcu zebranie wyników w postaci tabel i wykresów.

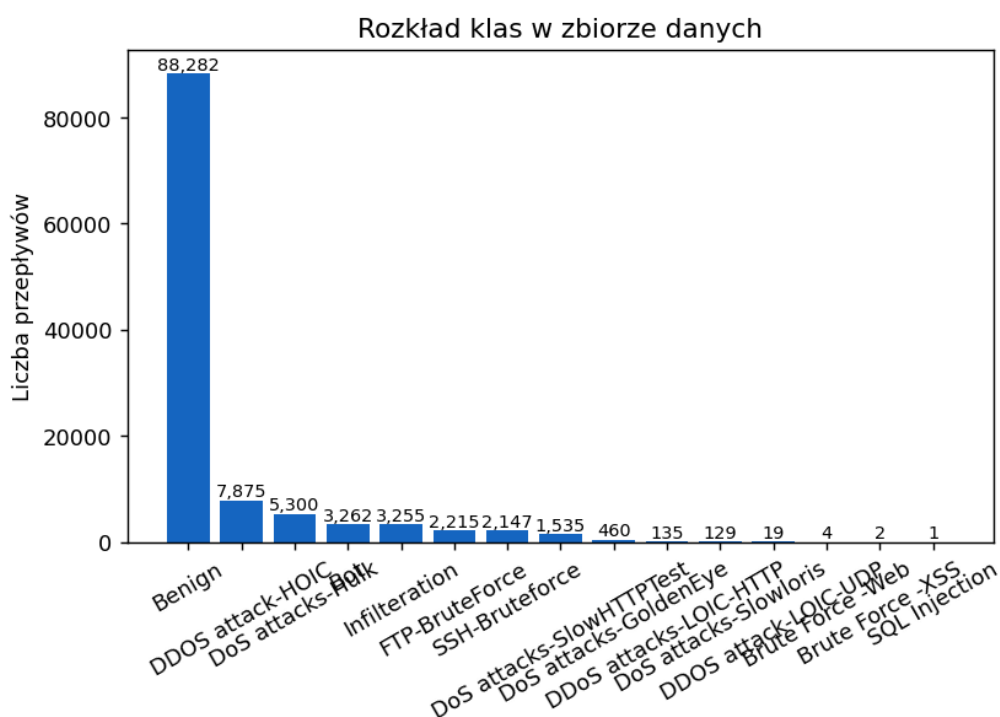
5.2 Zbiór danych

Głównym zbiorem danych jest CSE-CIC-IDS2018, duży, publicznie dostępny zbiór ruchu sieciowego zebranego na chmurowej infrastrukturze AWS przez Canadian Institute for

Cybersecurity. Zawiera on zarówno ruch normalny, jak i różne typy ataków (m.in. łamanie haseł, DoS/DDoS, ataki na aplikacje webowe, infiltrację oraz botnety). Każde połączenie opisane jest kilkudziesięcioma cechami liczbowymi, takimi jak czas trwania, liczba i rozmiar przesłanych pakietów, tempo transmisji czy liczniki flag sieciowych.

Pełny zbiór waży kilka gigabajtów (same pliki dzienne to łącznie około 7 GB), dlatego program wczytuje go w częściach i losowo próbuje, biorąc porcję danych z każdego dnia, ponieważ każdy dzień zawiera inne typy ataków. Ostatecznie wykorzystano wycinek liczący 114 621 połączeń opisanych 68 cechami i należących do 15 klas (ruch normalny oraz 14 typów ataków). Należy podkreślić, że jest to reprezentatywny wycinek połączonych plików, a nie pojedynczy plik ani komplet danych w całości.

Rysunek 1 przedstawia rozkład klas. Widoczna jest wyraźna przewaga ruchu normalnego nad atakami, typowa dla zadań wykrywania zagrożeń oraz duża rzadkość niektórych typów ataków (np. SQL Injection), co stanowi istotne wyzwanie dla modeli.



Rysunek 1. Rozkład klas w wykorzystanym wycinku zbioru CSE-CIC-IDS2018.

5.3 Przetwarzanie wstępne danych

Surowe dane wymagają uporządkowania. Etap przygotowania obejmuje:

- usunięcie kolumn nieprzydatnych do klasyfikacji (adresy IP, porty, znaczniki czasu) oraz kolumn o stałej wartości;
- poprawienie brakujących i błędnych wartości, np. wartości nieskończonych powstających przy dzieleniu;
- zamianę etykiet na prostą postać: ruch normalny lub atak;
- ujednolicenie skali cech, tak aby duże liczby nie zagłuszały mniejszych, co jest ważne zwłaszcza dla sieci neuronowych;
- podział danych na trzy części: uczącą, walidacyjną i testową.

Podział wykonano z ustalonym ziarnem losowości, dzięki czemu wyniki są w pełni powtarzalne. Liczności zbiorów wynoszą: 73 356 (uczący), 18 340 (walidacyjny) oraz 22 925 (testowy) połączeń.

Podział na trzy zbiory pełni ważną rolę metodologiczną. Zbiór uczący służy do dopasowania modelu, zbiór walidacyjny do kontrolowania przebiegu uczenia i decyzji o jego zatrzymaniu, a zbiór testowy, którego model nigdy nie widzi podczas treningu, do uczciwej, końcowej oceny. Podział jest stratyfikowany, co oznacza, że proporcje ruchu normalnego i ataków są w każdym z podzbiorów zbliżone do proporcji w całym zbiorze. Dzięki temu ocena nie jest zafałszowana przez przypadkowy nadmiar lub niedobór ataków w którymś z podzbiorów.

5.4 Konfiguracja modeli

Porównano pięć modeli o rosnącej złożoności:

- Regresja logistyczna: prosty model liniowy, traktowany jako punkt odniesienia; pokazuje, ile zyskujemy, sięgając po metody zaawansowane.
- Las losowy: zespół 300 drzew decyzyjnych z ważeniem klas.
- MLP: sieć z dwiema warstwami ukrytymi (128 i 64 neurony).
- CNN 1D: sieć konwolucyjna z dwiema warstwami filtrów.
- LSTM: dwukierunkowa sieć rekurencyjna.

Aby uczciwie potraktować rzadkość ataków, w każdym modelu zastosowano mechanizmy równoważące klasy, dzięki którym model jest mocniej karany za przeoczenie ataku. Sieci głębokie uczono z wykorzystaniem wczesnego zatrzymania: nauka kończy się, gdy model przestaje się poprawiać na danych walidacyjnych, co zapobiega przeuczeniu.

5.5 Środowisko i narzędzia

Całość zaimplementowano w języku Python. Modele klasyczne zbudowano przy użyciu biblioteki scikit-learn, a sieci głębokie w bibliotece PyTorch. Wykresy przygotowano w bibliotece matplotlib. Obliczenia prowadzono na komputerze Apple Macbook Pro z procesorem M3 Pro oraz 18GB pamięci RAM.

6. Implementacja programu

Niniejszy rozdział opisuje szczegóły implementacyjne programu napisanego na potrzeby eksperymentu. Całość powstała w języku Python i została podzielona na niezależne moduły o jasno określonej odpowiedzialności, co ułatwia zrozumienie kodu, jego testowanie oraz rozszerzanie o nowe modele i zbiory danych.

6.1 Organizacja kodu

Kod źródłowy znajduje się w katalogu src i jest zorganizowany w pakiety odpowiadające kolejnym etapom potoku przetwarzania. Strukturę katalogów przedstawia poniższy listing.

```
src/
  config.py          # parametry eksperymentu (dataklasy)
  utils.py           # ziarno losowości, pomiar czasu, zapis JSON
  data/
    synthetic.py     # generator danych syntetycznych
    download.py      # wczytywanie wycinka CSE-CIC-IDS2018
    loader.py        # wybór źródła danych (auto/cicids/synthetic)
    preprocess.py     # czyszczenie, kodowanie, podział, skalowanie
  models/
    classical.py     # las losowy i regresja logistyczna
    deep.py          # sieci MLP, CNN 1D, LSTM (PyTorch)
  evaluate.py        # obliczanie metryk
  train.py           # petla treningowo-ewaluacyjna
  plots.py           # generowanie wykresów
```

```
tables.py          # eksport tabel wynikow
run_all.py         # uruchomienie calego potoku
```

Listing 1. Struktura katalogów z kodem źródłowym.

6.2 Konfiguracja eksperymentu

Wszystkie parametry eksperymentu zebrano w module config w postaci klas danych (dataclass). Dzięki temu w jednym miejscu można zmienić źródło danych, liczbę drzew lasu losowego, liczbę epok uczenia sieci czy rozmiary warstw. Centralnie ustawione jest też ziarno losowości, które zapewnia powtarzalność wyników.

```
@dataclass
class RandomForestConfig:
    n_estimators: int = 300
    class_weight: str | None = "balanced"
    n_jobs: int = -1

@dataclass
class DeepConfig:
    epochs: int = 30
    batch_size: int = 256
    learning_rate: float = 1e-3
    hidden_dim: int = 128
    early_stopping_patience: int = 6
```

Listing 2. Fragment konfiguracji modeli (config.py).

6.3 Pozyskiwanie i przygotowanie danych

Moduł loader wybiera źródło danych zgodnie z konfiguracją: w trybie auto próbuje wczytać rzeczywisty zbiór, a w razie jego braku przełącza się na generator syntetyczny. Ponieważ pełny zbiór CSE-CIC-IDS2018 waży kilka gigabajtów, moduł download wczytuje pliki w częściach (chunkach) i losowo próbuje, dzięki czemu zużycie pamięci pozostaje ograniczone niezależnie od rozmiaru plików.

```
for chunk in pd.read_csv(path, chunksize=chunksize, low_memory=False):
    chunk = _normalise_columns(chunk)
    if 0.0 < sample_frac < 1.0:
        chunk = chunk.sample(frac=sample_frac, random_state=seed)
    buffer.append(chunk)
```

Listing 3. Strumieniowe, oszczędne pamięciowo wczytywanie danych (download.py).

Moduł preprocess realizuje czyszczenie danych: usuwa kolumny nieinformatywne i identyfikujące, zastępuje wartości nieskończone i brakujące, binaryzuje etykietę (ruch normalny lub atak), standaryzuje cechy i dzieli dane na zbiory uczący, walidacyjny oraz testowy w sposób stratyfikowany. Skaler dopasowywany jest wyłącznie na zbiorze uczącym, aby uniknąć wycieku informacji ze zbioru testowego.

6.4 Implementacja modeli

Modele klasyczne (las losowy i regresja logistyczna) oparto na bibliotece scikit-learn, a sieci głębokie zaimplementowano w PyTorch. Wszystkie modele udostępniają wspólny, prosty interfejs z metodami fit oraz predict_proba, dzięki czemu pętla treningowa traktuje je jednolicie. Trzy architektury sieci (MLP, CNN 1D, LSTM) dzielą wspólną klasę opakowującą, która zarządza pętlą uczenia, ważeniem klas i wczesnym zatrzymaniem.

```

class CNN1D(nn.Module):
    def __init__(self, in_dim, hidden, dropout):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv1d(1, 32, kernel_size=3, padding=1),
            nn.BatchNorm1d(32), nn.ReLU(), nn.MaxPool1d(2),
            nn.Conv1d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm1d(64), nn.ReLU(),
            nn.AdaptiveMaxPool1d(4))
        self.classifier = nn.Sequential(
            nn.Flatten(), nn.Linear(64 * 4, hidden),
            nn.ReLU(), nn.Dropout(dropout), nn.Linear(hidden, 1))

```

Listing 4. Definicja sieci konwolucyjnej CNN 1D (deep.py).

Niezbalansowanie klas uwzględniono na dwa sposoby: w lesie losowym przez parametr `class_weight` ustawiony na wartość `balanced`, a w sieciach głębokich przez ważenie klasy pozytywnej w funkcji straty. Uczenie sieci zatrzymywane jest automatycznie, gdy miara ROC-AUC na zbiorze walidacyjnym przestaje się poprawiać, co chroni przed przeuczeniem i skraca czas obliczeń.

6.5 Trening, ewaluacja i wizualizacja

Moduł `train` trenuje kolejno wszystkie modele na tym samym podziale danych, mierząc czas uczenia oraz czas klasyfikacji. Moduł `evaluate` oblicza komplet metryk (dokładność, precyzję, czułość, F1, ROC-AUC, PR-AUC) wraz z macierzą pomyłek. Moduły `plots` oraz `tables` zapisują wykresy i tabele wykorzystane w tym sprawozdaniu, dzięki czemu jego treść jest zawsze zgodna z faktycznymi wynikami.

6.6 Sposób uruchomienia

Cały eksperyment uruchamia się jednym poleceniem. Domyślnie program korzysta z danych rzeczywistych, jeśli są dostępne, a parametry można nadpisać argumentami wiersza poleceń.

```

# pelny eksperyment (dane rzeczywiste lub syntetyczne)
python -m src.run_all

# wymuszenie danych syntetycznych i krotszy trening
python -m src.run_all --source synthetic --epochs 10

# wygenerowanie sprawozdania w formacie .docx
python -m src.build_docx

```

Listing 5. Polecenia uruchamiające eksperyment i generujące sprawozdanie.

7. Wyniki i analiza

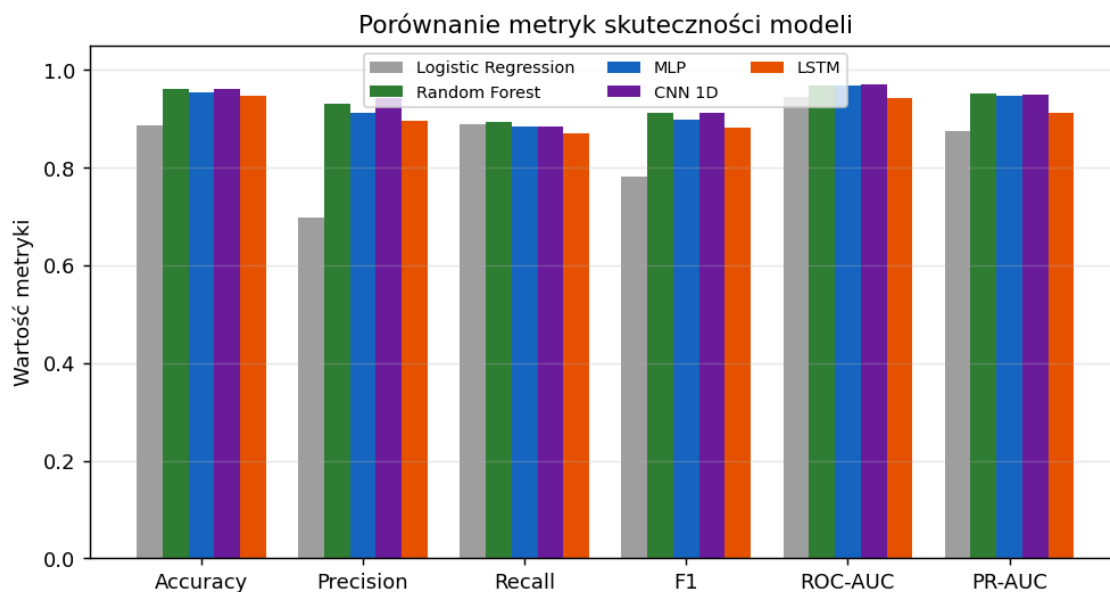
7.1 Skuteczność modeli

Tabela 1 zestawia metryki skuteczności wszystkich modeli na zbiorze testowym, a rysunek 2 przedstawia je graficznie.

Model	Dokładność	Precyzja	Czułość	F1	ROC-AUC	PR-AUC
Regresja logistyczna	0,886	0,699	0,890	0,783	0,945	0,875
Las losowy	0,961	0,932	0,893	0,912	0,968	0,953
MLP	0,954	0,912	0,884	0,898	0,967	0,946
CNN 1D	0,961	0,942	0,885	0,913	0,969	0,949
LSTM	0,947	0,895	0,870	0,882	0,943	0,913

Tabela 1. Skuteczność modeli na zbiorze testowym.

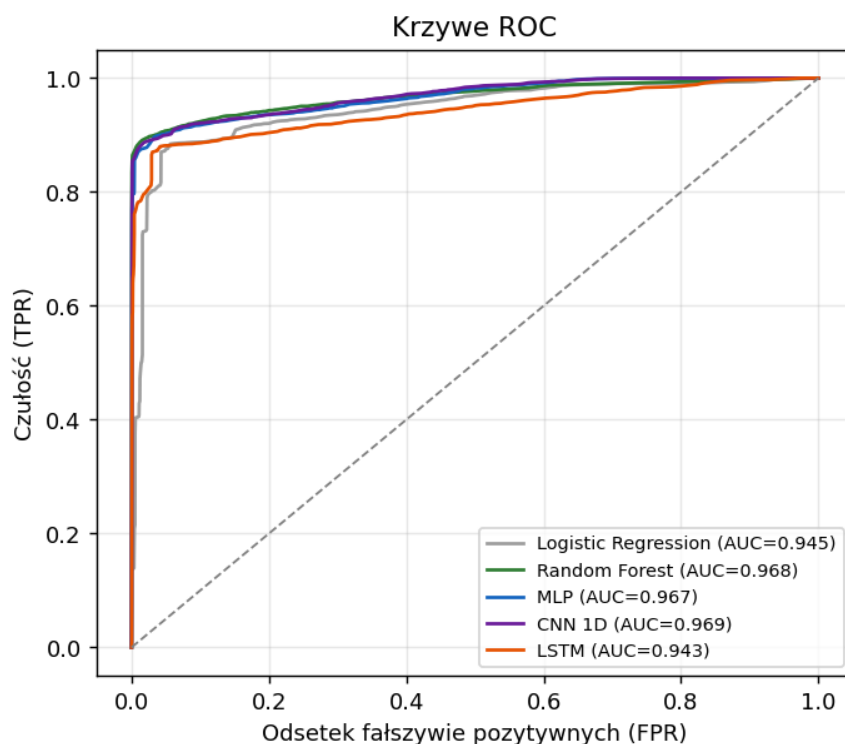
Z zestawienia wynika kilka obserwacji. Po pierwsze, wszystkie metody bardziej zaawansowane wyraźnie przewyższają prostą regresję logistyczną, co potwierdza, że atak zdradza dopiero kombinacja cech, a nie pojedyncze wartości. Po drugie, na szczycie panuje praktyczny remis: najlepszy model głęboki CNN 1D ($F1 = 0,913$) i las losowy ($F1 = 0,912$, $ROC-AUC = 0,968$) osiągają niemal identyczne wyniki. Po trzecie, oba modele mają nieco inny charakter: las losowy uzyskuje najwyższą czułość oraz najwyższy PR-AUC, podczas gdy CNN ma najwyższą precyzję. Najbardziej rozbudowany model, sieć LSTM, wypada najsłabiej spośród metod nieliniowych mimo największej złożoności.



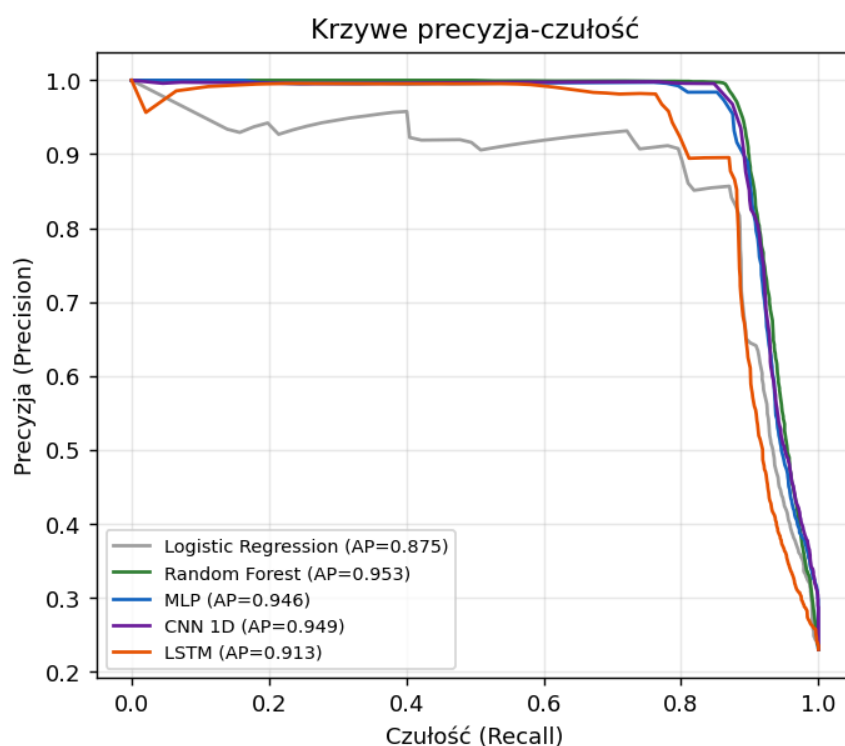
Rysunek 2. Porównanie metryk skuteczności modeli na zbiorze testowym.

7.2 Krzywe ROC i precyzja-czułość

Rysunki 3 i 4 przedstawiają krzywe ROC oraz precyzja-czułość. Krzywe dla wszystkich modeli nieliniowych przebiegają blisko lewego górnego rogu (ROC) i prawego górnego rogu (precyzja-czułość), co świadczy o dobrej separowalności klas. Regresja logistyczna wyraźnie odstaje, zwłaszcza na krzywej precyzja-czułość, która lepiej oddaje trudność zadania przy rzadkich atakach.



Rysunek 3. Krzywe ROC dla porównywanych modeli.



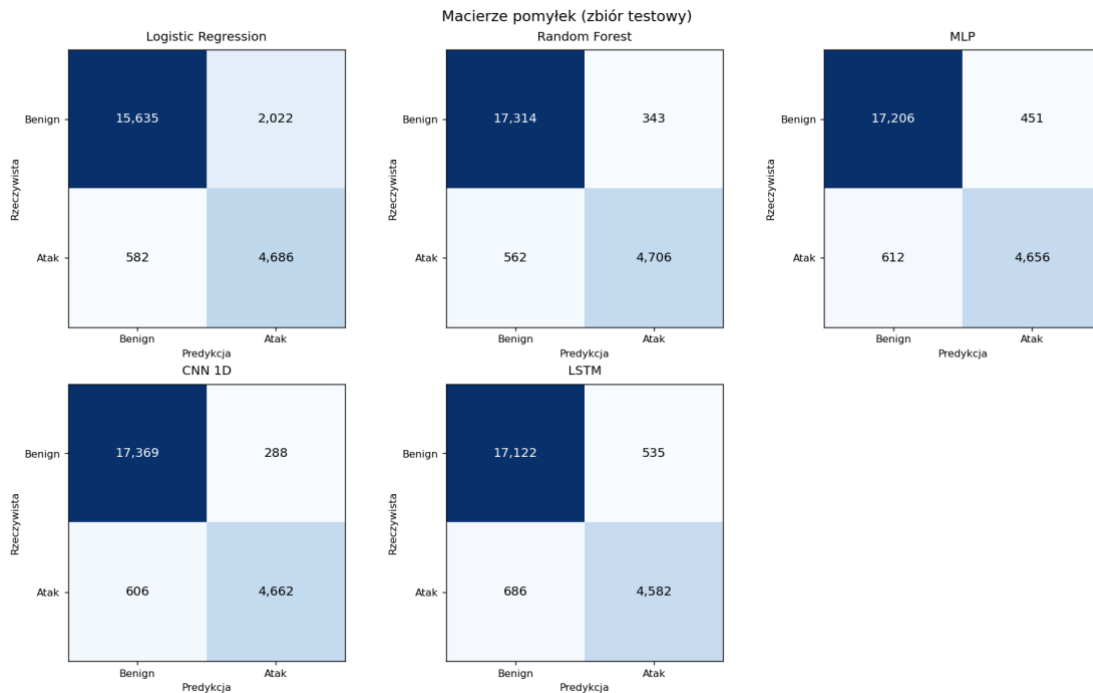
Rysunek 4. Krzywe precyzja-czułość dla porównywanych modeli.

7.3 Analiza błędów

Macierze pomyłek na rysunku 5 pozwalają zobaczyć, jakie błędy popełniają modele. Las losowy wychwytuje najwięcej ataków (najwyższa czułość), czyli rzadziej przepuszcza zagrożenie, co w kontekście ochrony danych jest cenne, bo przeoczone naruszenie kosztuje najwięcej. Sieć CNN nieco rzadziej podnosi fałszywy alarm (najwyższa precyzja), czyli mniej

obciąża zespół bezpieczeństwa niepotrzebnymi powiadomieniami. W praktyce punkt równowagi między tymi błędami można regulować progiem decyzyjnym, dopasowując go do polityki bezpieczeństwa organizacji.

Konkretne liczby dobrze ilustrują te różnice. Na zbiorze testowym las losowy przeoczył 562 ataków i wygenerował 343 fałszywych alarmów, sieć CNN przeoczyła 606 ataków przy zaledwie 288 fałszywych alarmach, a najsłabsza sieć LSTM przeoczyła 686 ataków. Widać więc, że las losowy najlepiej chroni przed najgroźniejszym błędem, czyli przepuszczeniem ataku, podczas gdy CNN najskuteczniej ogranicza liczbę niepotrzebnych powiadomień. Różnice te są jednak niewielkie w stosunku do całkowitej liczby ponad dwudziestu dwóch tysięcy testowanych połączeń.



Rysunek 5. Macierze pomyłek na zbiorze testowym dla wszystkich modeli.

7.4 Wydajność i koszt obliczeniowy

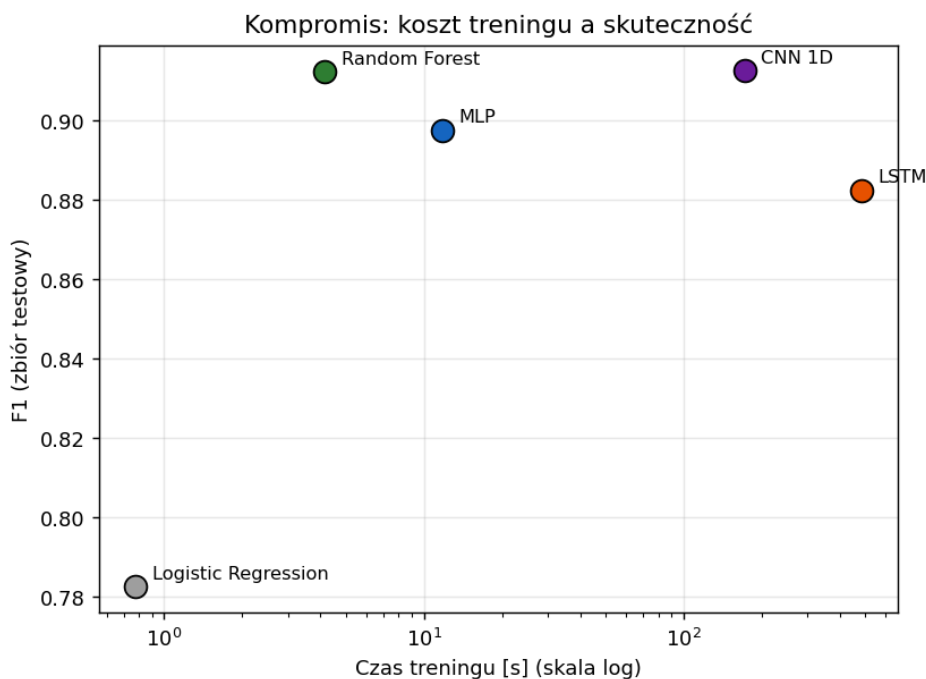
Skuteczność to tylko jeden wymiar oceny. W środowisku chmurowym, gdzie modele są często dotrenowywane i muszą klasyfikować ruch na bieżąco, kluczowy jest także koszt. Tabela 2 zestawia czasy treningu i inferencji oraz złożoność modeli.

Model	Czas treningu [s]	Czas inferencji [s]	Złożoność modelu
Regresja logistyczna	0,78	0,0006	69 wsp.
Las losowy	4,16	0,0801	300 drzew / 1 550 456 węzłów
MLP	11,82	0,0281	17 537 param.
CNN 1D	171,74	0,7103	39 553 param.
LSTM	487,45	3,9718	134 401 param.

Tabela 2. Wydajność i złożoność modeli.

I tu pojawia się sedno sprawy. Las losowy uczy się w zaledwie kilka sekund, podczas gdy najlepszy model głęboki potrzebuje kilkudziesięciu razy więcej czasu, by osiągnąć praktycznie ten sam wynik. Najbardziej rozbudowana sieć LSTM trenuje się ponad osiem minut, a mimo to wypada słabiej. Również w fazie działania las losowy i proste modele są wyraźnie szybsze od sieci CNN i LSTM. Kompromis między kosztem a skutecznością dobrze obrazuje rysunek

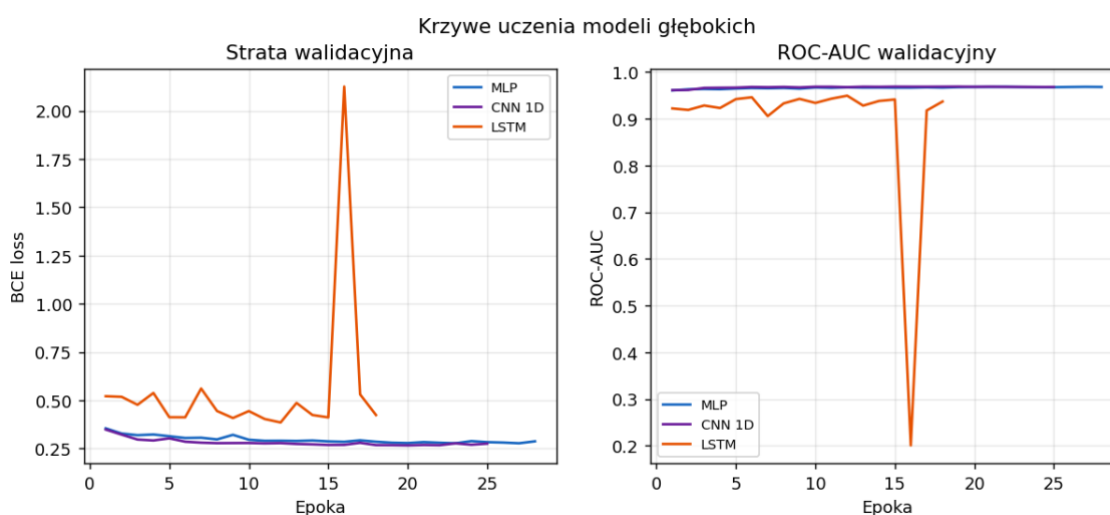
6: najbardziej opłacalne modele leżą w lewej górnej części wykresu, czyli osiągają wysoką skuteczność przy krótkim czasie uczenia.



Rysunek 6. Kompromis między czasem treningu (skala logarytmiczna) a skutecznością (F1).

7.5 Przebieg uczenia modeli głębokich

Rysunek 7 przedstawia krzywe uczenia modeli głębokich: stratę walidacyjną oraz ROC-AUC na zbiorze walidacyjnym w kolejnych epokach. Widoczna jest stabilna zbieżność i działanie mechanizmu wczesnego zatrzymania, który zapobiega przeuczeniu.

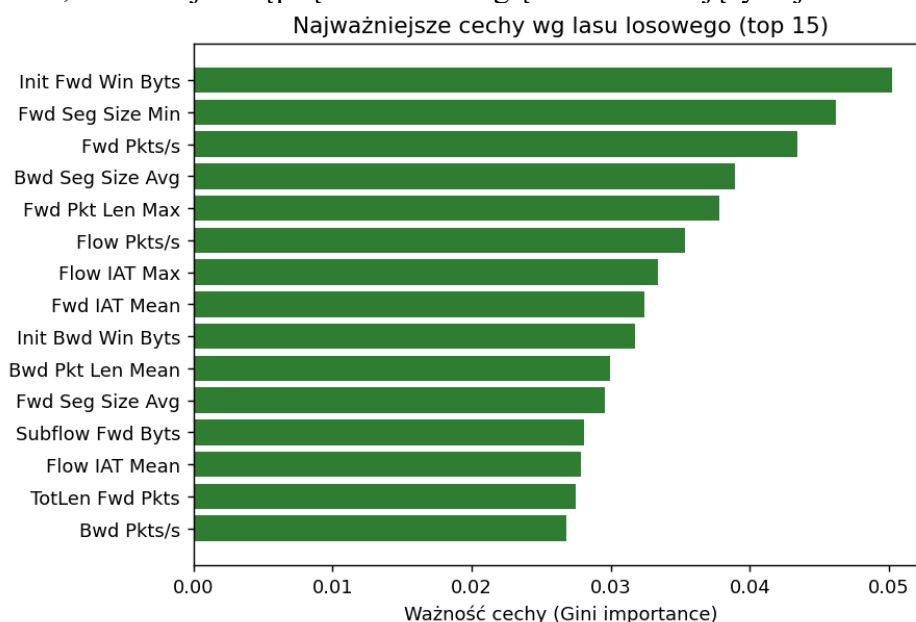


Rysunek 7. Krzywe uczenia modeli głębokich (strata i ROC-AUC walidacyjny).

7.6 Interpretowalność i ważność cech

Dodatkową zaletą lasu losowego jest interpretowalność. Rysunek 8 przedstawia najważniejsze cechy według miary ważności Giniego. Pozwala to zidentyfikować, które statystyki połączenia

najsilniej różnicują ruch złośliwy i normalny, co jest cenną informacją dla analityków bezpieczeństwa, a trudniej dostępną w modelach głębokich działających jak czarna skrzynka.



Rysunek 8. Najważniejsze cechy według lasu losowego (ważność Giniego).

Co ważne, najsilniej różnicujące cechy dotyczą wolumenu i tempa transmisji oraz statystyk czasu między pakietami, czyli wielkości, które w naturalny sposób odróżniają zautomatyzowane, masowe działania atakujących od ruchu generowanego przez człowieka. Zgodność tych obserwacji z intuicją ekspercką dodatkowo zwiększa zaufanie do modelu i pokazuje, że nie opiera on swoich decyzji na przypadkowych artefaktach zbioru danych.

7.7 Synteza wyników

Podsumowując, zebrane wyniki pozwalają wstępnie odpowiedzieć na postawione pytania badawcze. Pod względem skuteczności las losowy i najlepsza sieć są praktycznie nierozróżnialne, przy czym las losowy wykrywa najwięcej ataków. Pod względem kosztu las losowy jest zdecydowanie tańszy, oferując najlepszą relację skuteczność do nakładu obliczeniowego. Rozbudowane sieci sekwencyjne nie przyniosły przewagi, co oznacza, że większa złożoność nie przełożyła się na lepsze wyniki. Szczegółową interpretację tych obserwacji w kontekście wdrożeń chmurowych zawiera kolejny rozdział.

8. Dyskusja

8.1 Skuteczność a koszt w realiach chmury

Zestawienie skuteczności i kosztu prowadzi do najważniejszego spostrzeżenia tej pracy. W warunkach laboratoryjnych można pozwolić sobie na wielogodzinne treningi, jednak w chmurze sytuacja wygląda inaczej: modele bezpieczeństwa często wymagają regularnego dotrenowywania na świeżych danych, a czas obliczeń przekłada się wprost na koszt wynajmowanych zasobów. W tym świetle model osiągający niemal identyczną skuteczność kilkadziesiąt razy szybciej jest rozwiązaniem wyraźnie korzystniejszym ekonomicznie. Różnica w jakości między najlepszą siecią a lasem losowym jest na tyle mała, że w praktyce trudno ją odróżnić od szumu wynikającego z losowości danych, podczas gdy różnica w koszcie jest ogromna i jednoznaczna.

8.2 Interpretowalność i zaufanie

W bezpieczeństwie liczy się nie tylko trafność decyzji, lecz także możliwość jej wyjaśnienia. Analityk, który otrzymuje alarm, musi zrozumieć, dlaczego dane połączenie uznano za złośliwe, aby podjąć właściwą reakcję i uniknąć ślepego zaufania automatowi. Las losowy ma tu naturalną przewagę: udostępnia ranking ważności cech i pozwala prześledzić ścieżkę decyzji w drzewach. Sieci głębokie działają natomiast jak czarna skrzynka i wymagają dodatkowych, kosztownych technik wyjaśniania. Ten aspekt, choć trudny do ujęcia w pojedynczej metryce, ma duże znaczenie przy wdrożeniach produkcyjnych.

8.3 Ograniczenia danych tabelarycznych dla sieci głębokich

Uzyskane wyniki wpisują się w szerszą, znaną z literatury obserwację: na danych w postaci tabeli metody oparte na drzewach pozostają wyjątkowo konkurencyjne, a sieci głębokie nie wykorzystują swojej największej zalety, czyli zdolności do uczenia się reprezentacji z danych surowych. Cechy połączeń sieciowych są już bowiem starannie zaprojektowane przez ekspertów, więc dodatkowa moc sieci głębokich nie ma tu wiele do odkrycia. Tłumaczy to, dlaczego rozbudowane architektury sekwencyjne nie przyniosły przewagi, a najlepiej spośród sieci wypadły rozwiązania prostsze.

8.4 Rekomendacje wdrożeniowe

Na podstawie przeprowadzonej analizy można sformułować praktyczne rekomendacje. Jako domyślny wybór dla systemu wykrywania naruszeń w chmurze rozsądnie jest przyjąć las losowy: zapewnia on bardzo dobrą skuteczność, niski koszt treningu i działania, łatwość dotrenowywania oraz interpretowalność wspierającą pracę analityków. Sieć głęboką warto rozważyć jako uzupełnienie tam, gdzie priorytetem jest ograniczanie fałszywych alarmów i gdzie dostępne są akceleratory GPU obniżające koszt treningu. Niezależnie od wyboru modelu, próg decyzyjny powinien być dostrojony do polityki bezpieczeństwa: w środowiskach o wysokiej wrażliwości danych korzystniej jest preferować wyższą czułość kosztem większej liczby alarmów, natomiast tam, gdzie zespół jest niewielki, większą wagę przykładą się do precyzji. Dobrą praktyką jest również łączenie modelu uczenia maszynowego z klasycznymi regułami i monitorowaniem, tak aby decyzja nie opierała się na pojedynczym źródle.

9. Wnioski końcowe

9.1 Podsumowanie

W pracy zrealizowano empiryczne porównanie metod uczenia głębokiego i lasów losowych w zadaniu prognozowania naruszeń danych w chmurze, sformułowanym jako klasyfikacja połączeń sieciowych. Zbudowano kompletny, powtarzalny potok obejmujący przygotowanie danych, trening pięciu modeli oraz ich ocenę pod kątem skuteczności i kosztu. Najważniejsze wnioski są następujące:

- Obie rodziny metod skutecznie wykrywają naruszenia. Zarówno las losowy, jak i sieci głębokie osiągają wysokie wyniki, wyraźnie przewyższając prosty model liniowy. Złośliwa aktywność kryje się w złożonych zależnościach między cechami, które obie metody dobrze wychwytyją.
- Skuteczność jest niemal taka sama, ale koszt już nie. Najlepszy model głęboki (CNN 1D, $F1 = 0,913$) i las losowy ($F1 = 0,912$) osiągają praktycznie identyczną jakość, przy czym las losowy dodatkowo wykrywa najwięcej ataków, uczy się kilkadziesiąt razy szybciej i potrafi wskazać, które cechy zaważyły na decyzji.

- Bardziej skomplikowane nie znaczy lepsze. Rozbudowane sieci sekwencyjne, zwłaszcza LSTM, nie przewyższyły ani lasu losowego, ani prostszej sieci, generując jedynie wyższy koszt. Potwierdza to obserwację, że dla danych w postaci tabeli metody oparte na drzewach pozostają bardzo trudne do pobicia.
- Wybór metody zależy od kontekstu. Jeśli liczą się szybkie dotrenowywanie, niski koszt, dobra wykrywalność i zrozumiałość decyzji, las losowy jest naturalnym wyborem. Sieci głębokie warto rozważyć tam, gdzie zależy nam na maksymalnym ograniczeniu fałszywych alarmów i dysponujemy większymi zasobami obliczeniowymi.

9.2 Ograniczenia

Wyniki należy interpretować w świetle ograniczeń pracy. Eksperyment przeprowadzono na losowym wycinku danych ze względu na rozmiar pełnego zbioru; większa próbka mogłaby nieco zmienić konkretne liczby, choć raczej nie ogólne wnioski. Nie przeprowadzono również wyczerpującego strojenia hiperparametrów, lecz użyto rozsądnych wartości domyślnych, jednakowych dla wszystkich modeli. Obliczenia prowadzono na procesorze; na akceleratorach GPU względne czasy treningu sieci głębokich uległyby skróceniu, choć relacja kosztów pozostałaby jakościowo podobna.

9.3 Podsumowanie końcowe

Przeprowadzone badanie pokazuje, że w prognozowaniu naruszeń danych w chmurze nie ma jednej, bezdyskusyjnie najlepszej metody; wybór jest kompromisem między skutecznością a kosztem, szybkością i zrozumiałością decyzji. Dla danych opisujących ruch sieciowy las losowy okazuje się wyjątkowo silnym i tanim rozwiązaniem, któremu sieci głębokie co najwyżej dorównują, lecz go nie prześcigają, i to dopiero za cenę znacznie większego nakładu obliczeniowego. Płyne z tego praktyczny wniosek: zanim sięgnie się po skomplikowane sieci neuronowe, warto najpierw wypróbować dobrze dobraną metodę klasyczną.

10. Bibliografia

- [1] Breiman L., Random Forests, Machine Learning, 45(1), 5-32, 2001.
- [2] Hochreiter S., Schmidhuber J., Long Short-Term Memory, Neural Computation, 9(8), 1735-1780, 1997.
- [3] LeCun Y., Bottou L., Bengio Y., Haffner P., Gradient-Based Learning Applied to Document Recognition, Proceedings of the IEEE, 86(11), 2278-2324, 1998.
- [4] Sharafaldin I., Lashkari A. H., Ghorbani A. A., Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization, ICISSP, 108-116, 2018.
- [5] Moustafa N., Slay J., UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems, MilCIS, 1-6, 2015.
- [6] Tavallaee M., Bagheri E., Lu W., Ghorbani A. A., A Detailed Analysis of the KDD CUP 99 Data Set, IEEE CISDA, 1-6, 2009.
- [7] Grinsztajn L., Oyallon E., Varoquaux G., Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?, NeurIPS, 2022.
- [8] Goodfellow I., Bengio Y., Courville A., Deep Learning, MIT Press, 2016.
- [9] Pedregosa F. i in., Scikit-learn: Machine Learning in Python, JMLR, 12, 2825-2830, 2011.
- [10] Paszke A. i in., PyTorch: An Imperative Style, High-Performance Deep Learning Library, NeurIPS, 2019.